实验二题解

指针与引用的值修改操作

1. `modify1` 函数：实现引用的值修改

引用是变量的别名，通过引用可以直接修改原始变量的值。

```
void modify1(int &a) {
    a = 20;
}
```

2. `modify2` 函数：实现指针的值修改

指针存储的是变量的地址，通过指针可以间接修改原始变量的值。我们需要通过指针将传入的指针所指向的变量修改为指定的值

```
void modify2(int *a) {
    *a = 25;
}
```

3. `modify3` 函数：实现指针引用的值修改

指针引用是指针的引用，通过指针引用可以直接修改指针所指向的变量的值。我们需要通过指针引用将传入的指针所指向的变量修改为指定的值

```
void modify3(int *&a) {
    *a = 50;
}
```

交换操作

1. `swap1` 函数：实现引用的交换

引用是变量的别名，通过引用可以直接修改原始变量的值。函数参数是引用 `int &a` 和 `int &b`，可直接交换 `x1` 和 `y1` 的值。

```
void swap1(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

2. `swap2` 函数：实现指针的交换

指针存储的是变量的地址，通过指针可以间接修改原始变量的值。函数参数是指针 `int *a` 和 `int *b`，需要通过解引用操作符 `*` 交换 `x2` 和 `y2` 的值。

```
void swap2(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

3. `swap3` 函数：实现指针引用的交换

指针引用是指针的引用，通过指针引用可以直接修改指针所指向的变量的值。函数参数是指针引用 `int *&a` 和 `int *&b`，交换指针 `pa` 和 `pb` 的指向，从而间接交换 `a` 和 `b` 的值。

```
void swap3(int *&a, int *&b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

总结

- **引用**：可以直接交换两个变量的值。
- **指针**：通过解引用操作符 `*` 可以交换两个指针所指向的变量的值。
- **指针引用**：可以直接交换两个指针的指向，从而间接交换它们所指向的变量的值。

增量操作

1. `add1` 函数：实现引用的增量操作

引用是变量的别名，通过引用可以直接修改原始变量的值。我们需要通过引用将 `x` 和 `y` 增加指定的值。

```
void add1(int &a, int &b, int delta) {
    a += delta;
    b += delta;
}
```

2. `add2` 函数：实现指针的增量操作

指针存储的是变量的地址，通过指针可以间接修改原始变量的值。我们需要通过指针将 `x` 和 `y` 增加指定的值。

```
void add2(int *a, int *b, int delta) {
    *a += delta;
    *b += delta;
}
```

3. `add3` 函数：实现指针引用的增量操作

指针引用是指针的引用，通过指针引用可以直接修改指针所指向的变量的值。我们需要通过指针引用将 `x` 和 `y` 增加指定的值。

```
void add3(int *&a, int *&b, int delta) {
    *a += delta;
    *b += delta;
}
```

数组元素交换

1. `swapByIndex1` 函数：实现数组元素的引用交换

参数是数组 `int arr[]` 和两个索引 `i` 和 `j`。我们可以直接通过数组下标直接访问数组元素，并交换 `arr[i]` 和 `arr[j]` 的值。

```
void swapByIndex1(int arr[], int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
```

```
    arr[j] = temp;
}
```

2. `swapByIndex2` 函数：实现数组元素的指针交换

数组名本身是指向数组首元素的指针，我们同样可以通过指针操作来交换数组中指定索引的两个元素。

通过指针算术 `*(arr + i)` 访问数组元素，并交换 `*(arr + i)` 和 `*(arr + j)` 的值。注意到 `*(arr + i)` 等价于使用数组下标访问元素的方式，即 `arr[i]`。

```
void swapByIndex2(int *arr, int i, int j) {
    int temp = *(arr + i);
    *(arr + i) = *(arr + j);
    *(arr + j) = temp;
}
```

修改字符串内容

1. `modifyChar1` 函数：实现引用修改字符

引用是变量的别名，通过引用可以直接修改原始变量的值。

参数是引用 `char &c`，直接修改传入的字符为 `'a'`。

```
void modifyChar1(char &c) {
    c = 'a';
}
```

2. `modifyChar2` 函数：实现指针修改字符

指针存储的是变量的地址，通过指针可以间接修改原始变量的值。

参数是指针 `char *c`，通过解引用操作符 `*` 修改指针所指向的字符为 `'r'`。

```
void modifyChar2(char *c) {
    *c = 'r';
}
```

统计字符的出现次数

思路

我们开两个数组，一个数组 `assic` 存字符，一个 `count` 数组存这个字符出现了多少次，`assic[i]=ch` 表示第 `i` 个字符是 `ch` ， `count[i]` 表示第 `i` 个字符的次数。然后进行冒泡排序(任何排序算法都行)，注意如果出现次数相等，那么要把 `ascii` 值更大的放在前面。

注意这里的冒泡排序是针对我们使用的**两个数组**的，交换的时候要一起交换。同时注意是**多组输入。**

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>

int main()
{
    char s[1000];
    int count[256], i, j;
    int ascii[256];
    int t1, t2;
    while (scanf("%s", s) != EOF)
    {
        for (i = 0; i < 256; i++)
        {
            count[i] = 0;
            ascii[i] = i;
        }
        for (i = 0; i < strlen(s); i++)
        {
            count[s[i]] ++;
        }
        // sort
        for (j = 0; j < 255; j++)
        {
            for (i = 0; i < 255 - j; i++)
            {
                if (count[i] < count[i + 1])
                {
                    t1 = count[i];
                    count[i] = count[i + 1];
                    count[i + 1] = t1;
                    t2 = ascii[i];
                    ascii[i] = ascii[i + 1];
```

```
                    ascii[i + 1] = t2;
                }
                else if (count[i] == count[i + 1])
                {
                    if (ascii[i] < ascii[i + 1])
                    {
                        t2 = ascii[i];
                        ascii[i] = ascii[i + 1];
                        ascii[i + 1] = t2;
                    }
                }
            }
        }
        // output
        for (i = 0; i < 256; i++)
        {
            if (count[i] == 0) break;
            printf("%c:%d ", ascii[i], count[i]);
        }
        printf("\n");
    }

    return 0;
}
```

三角形

思路

先对数组进行排序，然后从大的往小的方向进行遍历，判断相邻的三个数a, b, c，看看是否存在$b + c > a$。（排序后从大到小进行遍历，就保证了如果$b + c$都不大于a，那么数组里面就不存在其它两个数字大于a了）。

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>

void sort(int a[], int n)
{
    int i, j, tmp;
    for (j = 0; j < n - 1; j++)
    {
        for (i = 0; i < n - 1 - j; i++)
```

```
        {
            if (a[i] > a[i + 1])
            {
                tmp = a[i];
                a[i] = a[i + 1];
                a[i + 1] = tmp;
            }
        }
    }
    return;
}

int main()
{
    int n, i, f;
    int a[1005];
    while (scanf("%d", &n) != EOF)
    {
        f = 0;
        for (i = 0; i < n; i++)
        {
            scanf("%d", &a[i]);
        }
        sort(a, n);
        for (i = n - 1; i >= 2; i--)
        {
            if (a[i - 1] + a[i - 2] > a[i])
            {
                f = 1;
                printf("%d\n", a[i - 1] + a[i - 2] + a[i]);
                break;
            }
        }
        if (f == 0)printf("-1\n");
    }
    return 0;
}
```

混合比较

思路

使用`%d`输入输出十进制，`%x`对应十六进制，`%o`对应八进制，之后用“打擂台”方法求最大值。

读入以后存在程序里的就都是数值了，因此可以直接进行比较，但是题目要求输出时保留原来的进制，所以要另外使用一个变量`int type`来记录最大值的进制。

代码可以用三个循环实现，不必使用数组，核心部分（擂台）：

```
if(num > max_num):
    max_num = num;
    type = ...;
```

给`type`赋的值取决于当前输入`num`的进制，可以用123表示三个进制。

注意这里使用的是小于号，这样满足题目要求即大小相等时取第一个出现的数，如果改成大于等于号则会取到最后一个出现的数。

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    int x, y, z;
    int a, max_num = -1, type;
    scanf("%d%d%d", &x, &y, &z);
    for(int i = 0; i < x; i++)
    {
        scanf("%d", &a);
        if(a > max_num)
            max_num = a, type = 0;
    }
    for(int i = 0; i < y; i++)
    {
        scanf("%o", &a);
        if(a > max_num)
            max_num = a, type = 1;
    }
    for(int i = 0; i < y; i++)
    {
        scanf("%x", &a);
        if(a > max_num)
            max_num = a, type = 2;
    }
    if(type == 0)
        printf("%d\n", max_num);
    else if(type == 1)
```

```
        printf("%o\n", max_num);
    else
        printf("%x\n", max_num);
}
```

五子棋裁判

思路

本题主要是代码实现比较麻烦，随时保持代码整洁对提高正确率很有帮助。

其中一种可能的写法：

首先定义一个二维字符数组`s[][]`存放棋盘数据。

之后用二重循环遍历棋盘的每个棋子(i,j)，再加一个次数为5的循环$(0<=k<5)$，遍历查找连子，水平方向的连子坐标为$(i,j+k)$，垂直$(i+k,j)$，主对角线$(i+k,j+k)$，副对角线$(i+k,j-k)$，在这个过程记录连子的数量。

在遍历过程中要防止坐标超过棋盘的范围，超过范围时提前用 break 退出本次循环。

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#define MAX_SIZE (21)

int main()
{
    int t = 0;
    scanf("%d", &t);
    while (t--)
    {
        int n;
        char s[MAX_SIZE][MAX_SIZE];

        scanf("%d", &n);
        for (int i = 0; i < n; i++)
            scanf("%s", s[i]);

        int b_win = 0;
        int w_win = 0;

        for(int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
```

```
        {
            // 记录黑白连子的数量
            int b_cnt, w_cnt;

            // 水平（i, j + k）
            b_cnt = 0, w_cnt = 0;
            for (int k = 0; k < 5; k++)
            {
                if (j + k >= n) break;
                if (s[i][j + k] == 'W') w_cnt++;
                if (s[i][j + k] == 'B') b_cnt++;
            }
            if (b_cnt == 5) b_win = 1;
            if (w_cnt == 5) w_win = 1;

            // 垂直（i + k, j）
            b_cnt = 0, w_cnt = 0;
            for (int k = 0; k < 5; k++)
            {
                if (i + k >= n) break;
                if (s[i + k][j] == 'W') w_cnt++;
                if (s[i + k][j] == 'B') b_cnt++;
            }
            if (b_cnt == 5) b_win = 1;
            if (w_cnt == 5) w_win = 1;

            // 主对角线：（i + k, j + k）
            b_cnt = 0, w_cnt = 0;
            for (int k = 0; k < 5; k++)
            {
                if (i + k >= n || j + k >= n) break;
                if (s[i + k][j + k] == 'W') w_cnt++;
                if (s[i + k][j + k] == 'B') b_cnt++;
            }
            if (b_cnt == 5) b_win = 1;
            if (w_cnt == 5) w_win = 1;

            // 副对角线：（i + k, j - k）
            b_cnt = 0, w_cnt = 0;
            for (int k = 0; k < 5; k++)
            {
                if (i + k >= n || j - k < 0) break;
                if (s[i + k][j - k] == 'W') w_cnt++;
                if (s[i + k][j - k] == 'B') b_cnt++;
            }
            if (b_cnt == 5) b_win = 1;
            if (w_cnt == 5) w_win = 1;
        }
```

```
        if (b_win && w_win) printf("???\n");
        else if (b_win)
            printf("Black\n");
        else if (w_win)
            printf("White\n");
        else
            printf("Not so fast\n");
    }
}
```

创建链表

```
node* createList()
{
    // 使用尾部插点法构造链表
    node* head = NULL;
    node* p1, * p2; // 维护两个指针，p1用来开辟新的结点，p2用来指向当前链表的尾部
    p1 = new node;
    p2 = p1;
    cin >> p1->val;
    while (p1->val != -1)
    {
        if (head == NULL)
            head = p1;
        else p2->next = p1; // 在链表尾部插入新结点
        p2 = p1; // 更新指向链表尾部的指针p2
        p1 = new struct node; // p1用来开辟新的结点
        cin >> p1->val;
    }
    p2->next = NULL; // 尾部的指针为空，作为链表结束的标记
    delete p1;
    return head;
}
```

逆序链表

```
node* createList()
{
    node* head = NULL;
```

```
    node* p1, * p2;
    p1 = new node;
    p2 = p1;
    cin >> p1->val;
    while (p1->val != -1)
    {
        if (head == NULL)
            head = p1;
        else p2->next = p1;
        p2 = p1;
        p1 = new struct node;
        cin >> p1->val;
    }
    p2->next = NULL;
    delete p1;
    return head;
}

node* reverseList(node* head)
{
    // 使用头部插点法构造新链表
    // 只是改变旧链表的结点的顺序（逆序），没有生成新结点
    node* newHead = NULL; // 新链表的头部指针
    node* cur = head;  // 遍历旧链表的指针
    while (cur)
    {
        node* next = cur->next;
        cur->next = newHead; // 新插入的结点总是放在newHead指向的结点之前
        newHead = cur; // 更新newHead
        cur = next;    // 遍历旧链表
    }
    return newHead;
}
```

删除元素

```
node* createList()
{
    node* head = NULL;
    node* p1, * p2;
    p1 = new node;
    p2 = p1;
    cin >> p1->val;
    while (p1->val != -1)
```

```
    {
        if (head == NULL)
            head = p1;
        else p2->next = p1;
        p2 = p1;
        p1 = new struct node;
        cin >> p1->val;
    }
    p2->next = NULL;
    delete p1;
    return head;
}

node* deleteElements(node* head, int v)
{
    // 尾部插点法构造新链表（即删除值为v结点后的链表）
    // 新链表的结点都是从旧链表中获取，没有生成新结点

    node* newHead = new node; //新链表的头结点，不存数据，和头指针的意义不同
    node* tail = newHead;     // 新链表的尾部指针
    node* cur = head; // 遍历旧链表的指针
    while (cur)
    {
        node* next = cur->next;
        if (cur->val == v)
        {
            delete cur;      // 删除值为v的结点
        }
        else
        {
            tail->next = cur; // 在新链表的尾部插入节点
            tail = tail->next; // 更新尾部指针
        }
        cur = next;
    }
    tail->next = NULL; // 新链表的尾部置为NULL
    node* next = newHead->next; // 新链表中仍然使用头指针，不使用头结点
    delete newHead; // 删掉头结点
    return next;
}
```